

Abstract

This study introduces *NTUGR*, an advanced global routing algorithm aimed at optimizing large-scale 3D routing challenges, including wirelength, via, and overflow costs for designs with over 200 million pins. Leveraging multi-threading and enhanced routing strategies, *NTUGR* notably improves cost efficiency and computational speed, outperforming existing solutions like *GAMER* in various tests. This contribution not only advances physical design automation but also enriches our comprehension of global routing complexities, suggesting future exploration in scalable solutions and innovative techniques like reinforcement learning and GPU acceleration for routing optimization.

Keywords: Global Routing, Physical Design, Multi-threading, Dynamic Net Separation

Table of Contents

Abstract (Chinese)	i
Abstract	ii
Chapter 1. Introduction	1
1.1 Global Routing	2
1.2 Motivation	3
1.3 Related Works	4
1.3.1 Initial Routing	4
1.3.2 Maze Routing	5
1.3.3 GPU-Accelerated Global Routing	5
1.3.4 ML-Enhanced Global Routing	6
1.4 Our Contributions	7
1.5 Thesis Organization	7
Chapter 2. Preliminaries	8
2.1 Technical Terms	8
2.1.1 Capacity	8
2.1.2 Demand	9
2.1.3 Resource	9
2.2 Problem Formulation	10
Chapter 3. Global Routing Algorithm	12
3.1 Preprocessing	14
3.1.1 Selection of Pin Access Points	14
3.1.2 Determining the Order of Nets	14

3.1.3	Optimized Net Distribution for Parallel Processing	15
3.2	Multi-Threaded Pattern Routing	16
3.2.1	Optimization of Wirelength through Steiner Tree Construction . .	16
3.2.2	Construction of the Routing Directed Acyclic Graph	17
3.2.3	DAG-based Pattern Routing	17
3.3	Congestion-Aware Pattern Routing	19
3.3.1	Construct Congestion Map	19
3.3.2	Construct Detours	19
 Chapter 4. Experimental Results		21
4.1	Experimental Setup	21
4.2	Results and Comparisons	22
 Chapter 5. Conclusions and Future Works		24
5.1	Conclusions	24
5.2	Future Works	26
5.2.1	Pattern Routing	26
5.2.2	Maze Routing	26
 Bibliography		27

Chapter 1

Introduction

This work introduces a comprehensive global router designed to tackle the challenges of large-scale global routing, utilizing simplified input/output formats as specified by the International Symposium on Physical Design (ISPD) 2024 Contest [6]. We begin by elucidating the concept of global routing and its critical role in modern circuit design in Section 1.1. Following this, our motivation for addressing the complexities associated with large-scale global routing is detailed in Section 1.2. A review of pertinent literature and previous approaches to similar problems is discussed in Section 1.3. We then outline our contributions to the field in Section 1.4. The document concludes with an overview of the structure and organization of this work, presented in Section 1.5, guiding the reader through the subsequent chapters.

1.1 Global Routing

In VLSI (Very Large Scale Integration) circuit design, global routing is crucial, shaping the preliminary pathways for chip connections and substantially influencing performance metrics like power usage and overall efficiency. This stage outlines broad routing regions for each net, laying the groundwork for the subsequent detailed routing phase that specifies the exact wire geometries. As the scale and complexity of chip designs increase, conventional global routing techniques struggle with scalability and runtime, which affects their practicality for large-scale designs. Enhancing these routing methods is vital, given their impact on various design stages listed in Table 1.1, and innovation in global routing is imperative to resolve these challenges without degrading design quality.

Table 1.1: Application of global routing across various VLSI design stages.

Usage	Design Size	Runtime	Congestion Resolving Effort	Global Router Features
Logic Synthesis	20M-100M	fast	low	routability estimation, timing estimation
Physical Planning	20M-100M	fast	low	routability estimation, timing estimation
Placement	1M-5M	fast	low	routability estimation, timing estimation
Optimization	1M-5M	medium	medium	timing-driven topology, buffering-friendly topology, incremental update
Guide Detailed Routing	1M-5M	long	high	timing-driven topology, power-driven

1.2 Motivation

The necessity to innovate in large-scale global routing is driven by its critical impact on the performance and power efficiency of VLSI circuits. With the increasing complexity of integrated circuits, existing routing algorithms struggle to balance expediency and quality—often yielding either suboptimal routes or suffering from extensive runtimes unsuitable for large-scale designs. This research is motivated by the urgent need for a novel global routing solution that encapsulates the advantages of current methods while mitigating their shortcomings. By amalgamating the strengths of various approaches, the aim is to develop a new global router that delivers high-quality routing solutions efficiently, aligning with the evolving demands of VLSI design and propelling the hardware innovation forward.

1.3 Related Works

This section offers an in-depth examination of the advancements in global routing methodologies, tracing the lineage from foundational techniques to contemporary innovations that leverage GPU and ML technologies. Each subsection not only explores individual contributions but also situates them within the broader ecosystem of electronic design automation, highlighting the cumulative progress and the synergistic effects of these technologies.

1.3.1 Initial Routing

Initial routing strategies serve as the cornerstone for effective global routing, establishing a framework upon which subsequent refinements are built. The concept of pattern routing, as detailed by Kastner et al. [5], introduced a paradigm shift with the L or Z shaped patterns, enhancing predictability and mitigating coupling issues. This method laid the groundwork for FastRoute’s [11] congestion-driven Steiner trees and edge-shifting techniques, which further refined routing efficiency by dynamically adjusting to congestion levels. The introduction of CUGR [4] and its evolution, EDGE [3], marked a significant advancement in routing topology optimization. By leveraging dynamic programming and DAG-based methodologies, these works offered novel insights into routing exploration and congestion management, illustrating a trend towards more complex and efficient initial routing frameworks.

1.3.2 Maze Routing

Maze routing represents a critical evolution beyond initial routing, introducing sophisticated negotiation-based strategies to address rip-up and reroute challenges. FastRoute 2.0 [12] and the techniques developed by Liu et al. [9, 10] pushed the boundaries of maze routing efficiency. By implementing limitations on search space and introducing a multithreading approach, these innovations aimed to optimize the delicate balance between runtime and routing quality. Notably, NCTU-GR 2.0 [9] demonstrated the effectiveness of handling overlapping bounding boxes through multithreaded routing, showcasing significant reductions in runtime. These advancements underscore a focused effort to refine maze routing techniques, striving for efficiency without compromising on the quality of routing outcomes.

1.3.3 GPU-Accelerated Global Routing

The integration of GPU acceleration into global routing has revolutionized the field, enabling unprecedented levels of efficiency and quality. The studies by Lin et al. [7, 8] exemplify the transformative impact of GPU technologies, demonstrating significant reductions in runtime alongside improvements in routing quality. By harnessing the parallel processing power of GPUs, these works have opened new avenues for tackling the increasing complexity of global routing challenges, positioning GPU acceleration as a cornerstone of modern electronic design automation strategies.

1.3.4 ML-Enhanced Global Routing

The advent of machine learning has introduced a new dimension to global routing, offering innovative approaches to longstanding challenges. Zhou et al.'s MEDUSA [13] and Liu et al.'s REST [2] represent pivotal contributions to this domain, leveraging ML to refine congestion estimation and optimize Rectilinear Steiner Minimum Trees, respectively. These methodologies not only surpass traditional techniques in efficiency and accuracy but also illustrate the potential of ML to revolutionize routing strategies. By integrating ML into global routing, researchers are unlocking novel solutions to complex optimization problems, heralding a new era of innovation in electronic design automation.

In expanding upon these subsections, we weave a narrative that illustrates the intricate evolution of routing methodologies, from their inception to the cutting-edge integrations of GPU and ML technologies. This comprehensive overview not only highlights individual contributions but also underscores the collaborative progression towards more sophisticated, efficient, and accurate global routing solutions.

1.4 Our Contributions

In this work, we address the global routing problem with the objective of minimizing wirelength, via, and overflow costs for large-scale instances. Our contributions are summarized as follows:

- We introduce *NTUGR*, a global router employing multi-threading techniques to accelerate the routing process.
- *NTUGR* is compatible with the simplified input/output formats as defined by the International Symposium on Physical Design (ISPD) 2024 Contest [6], capable of efficiently handling cases with over 200 million pins.
- While *NTUGR* shares its core engine with CUGR2 [3], it incorporates more generalized patterns and eliminates redundant elements, achieving superior speed and performance.
- We propose a novel dynamic net separation framework that efficiently distributes nets for parallel processing, enhancing algorithmic efficiency.
- Experimental results demonstrate that *NTUGR* significantly outperforms the current state-of-the-art, GAMER [8], in terms of total wirelength, via, overflow costs, and computation time.

1.5 Thesis Organization

The remainder of this work is organized as follows. Chapter 2 defines the technical terminology and formulates the problem addressed in this study. Chapter 3 describes our global routing methodology, detailing each step of the process. Chapter 4 discusses the experimental results obtained from our studies. Finally, Chapter 5 concludes our work and proposes the future works.

Chapter 2

Preliminaries

In this chapter, we first define the technical terms employed throughout this paper in Section 2.1, followed by the formulation of the global routing problem in Section 2.2.

2.1 Technical Terms

In the development of our algorithm, we rely on three fundamental concepts: capacity, demand, and resource. These terms apply to both wire edges and global routing cells (GCells), serving as the backbone for our routing strategy. Below is a concise overview of each term:

2.1.1 Capacity

Capacity, denoted as c , refers to the maximum number of nets or tracks that can traverse a given element (either an edge or a GCell). For a wire edge $e(u, v)$, the capacity $c(u, v)$ represents the number of tracks through the edge, essentially the limit on nets usage. For a GCell u , its capacity $c(u)$ is the average capacity of its adjacent wire edges, providing a metric for its throughput capability.

2.1.2 Demand

Demand, symbolized by d , quantifies the portion of capacity currently occupied by existing nets. For an edge, $d(u, v)$ includes both the wires running through it and the impact of vias on its capacity. Specifically, it combines the count of wires $wire(u, v)$ and an estimated tracks usage by vias $vias(u)$. For a GCell, $d(u)$ amalgamates the wire demand from adjoining edges and the via demand within the cell, reflecting the current load.

2.1.3 Resource

Resource, indicated as r , represents the remaining unoccupied capacity, highlighting the potential for additional routing within an edge or a GCell. Calculated as the capacity minus the demand, the resource value $r(u, v)$ for an edge or $r(u)$ for a GCell offers insights into the available routing space and congestion levels. This measure aids in identifying areas with sufficient capacity margin for future routing activities.

These definitions underpin our algorithm's approach to efficiently managing routing paths and congestion across the network.

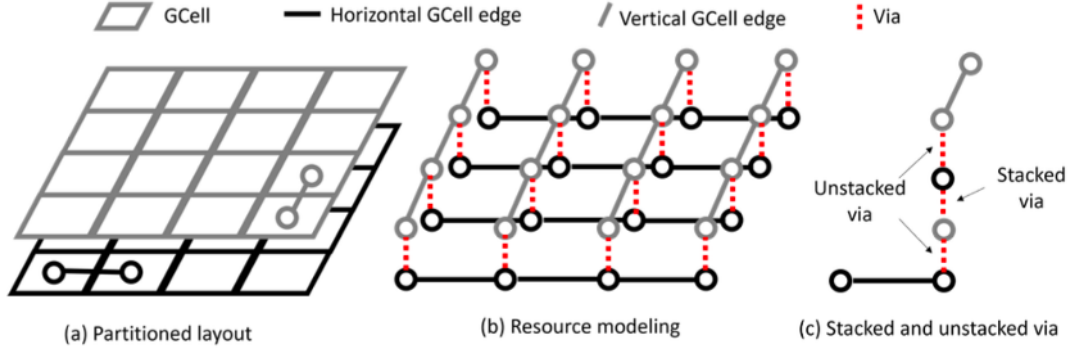


Figure 2.1: Illustration of a GCell grid graph.

2.2 Problem Formulation

Global routing operates in a 3D space partitioned into global routing cells (GCells), defined by intersecting horizontal and vertical lines, as shown in Figure 2.1(a). This setup creates a grid graph $G(V, E)$, where GCells act as vertices ($v \in V$) and edges ($e \in E$) link adjacent GCells across layers, depicted in Figure 2.1(b). Each layer favors a specific routing direction, and circuit data is captured in two key input files: a .cap file for routing resources and a .net file for pin information. The primary objective is to connect pins across nets without causing overflow, minimizing both wire length and via count.

Global routing leverages the GCell coordinate system atop metal layers to optimize pin accessibility and routing paths. It strategically avoids using Metal 1 for net paths, favoring Metal 2 connections via vias instead. Stacked vias, which traverse multiple layers, diminish edge capacity at GCells, necessitating a compensatory increase in routing demand by half a track for those edges, as illustrated in Figure 2.1(c). In contrast, unstacked vias' impact is directly integrated into the routing strategy without additional demand adjustments.

Figure 2.2 presents a simple yet effective global routing example, where wires

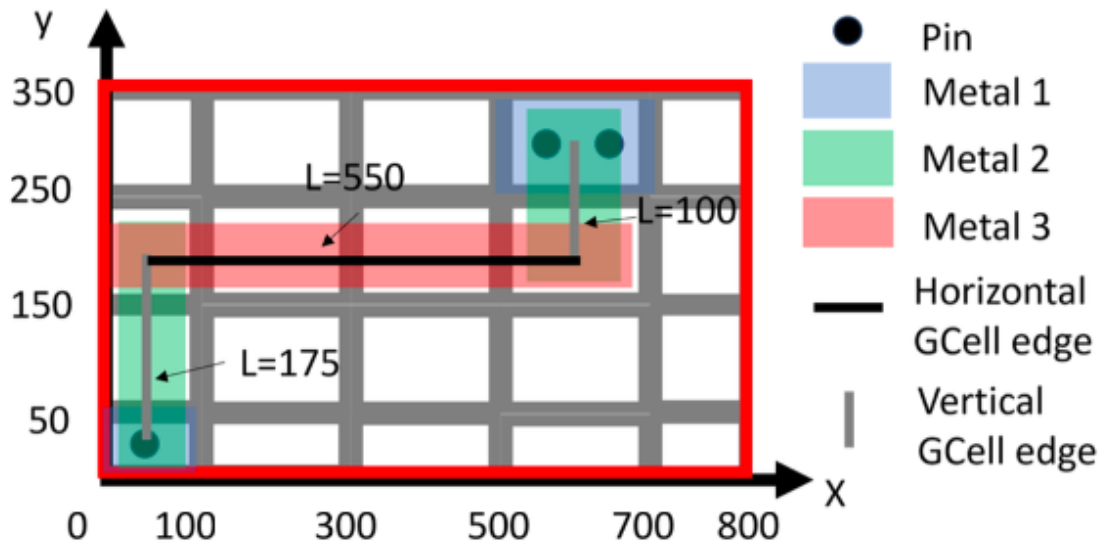


Figure 2.2: An example of a global routing solution.

create a connected network enveloping all net pins. Connectivity is ensured through shared metal layers or via links, as depicted. A successful global routing solution must satisfy validity criteria for all circuit nets.

The evaluation of global routing solutions focuses on:

1. Total wire length across all signal nets,
2. The number of vias employed,
3. Overflow levels, indicating routing demand surpassing capacity,
4. Routing execution time, and
5. The overall connectivity and validity of the net routing solution.

Chapter 3

Global Routing Algorithm

In this chapter, we provide a comprehensive overview of the algorithm’s framework. Illustrated in Figure 3.1, the framework of our algorithm is delineated across three pivotal stages: (1) Preprocessing, (2) Multi-Thread Pattern Routing, and (3) Congestion-Aware Pattern Routing. The initial stage, Preprocessing, focuses on preparing nets for routing by simplifying their structures and organizing their processing sequence. Subsequently, the Multi-Thread Pattern Routing stage adopts a multi-threaded strategy to attain an initial routing outcome. In the final stage, nets that exceed capacity constraints are rerouted using a congestion-aware pattern routing method. Detailed explanations of these stages are provided in Sections 3.1, 3.2, and 3.3, respectively.

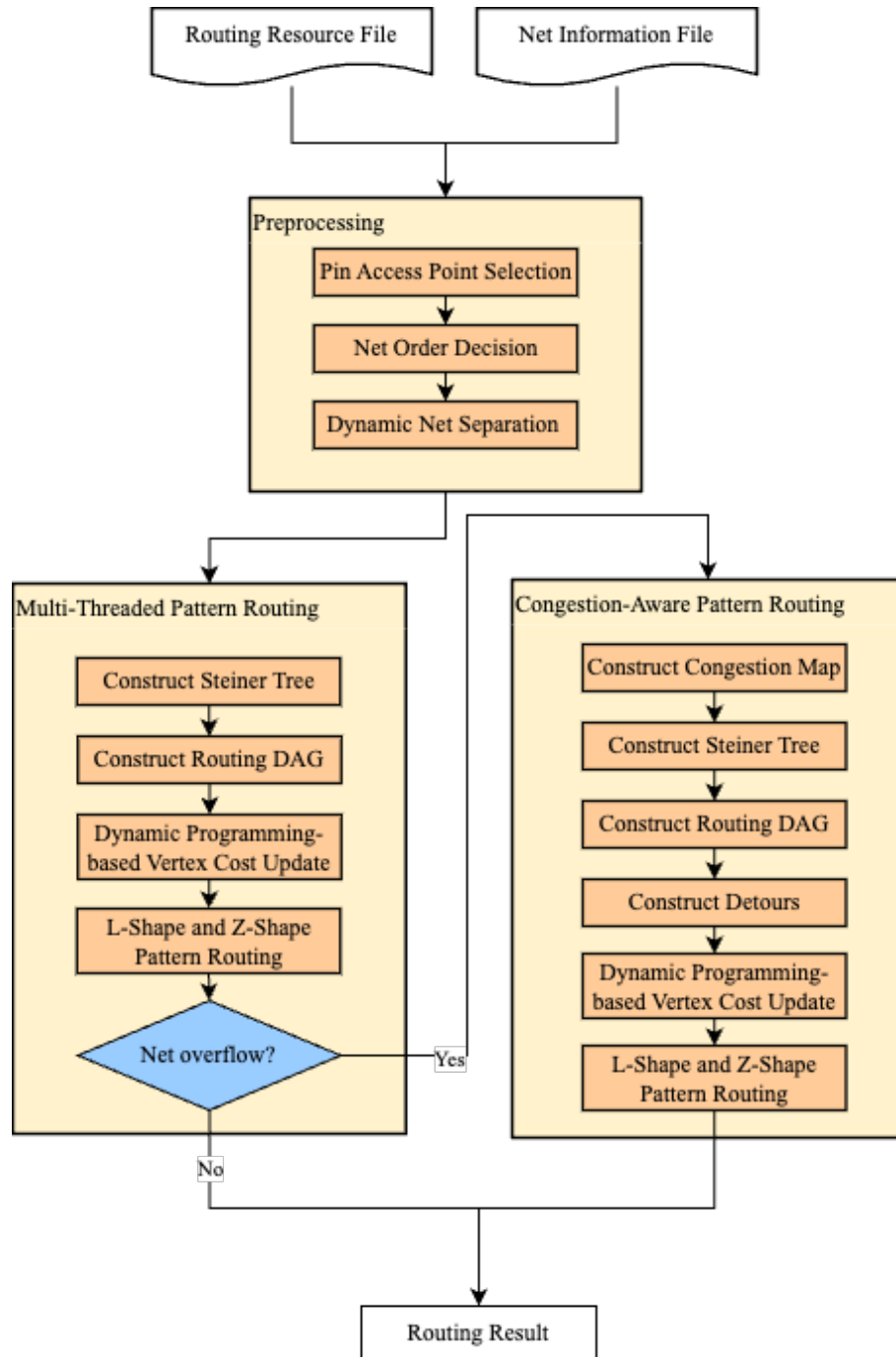


Figure 3.1: Our global routing algorithm flow.

3.1 Preprocessing

In this phase, we utilize two critical input files: a routing resource file (.cap) and a net information file (.net), which together provide all necessary information for global routing. Resource preprocessing entails constructing a multi-layered map, to precisely monitor grid capacity. For net preprocessing, strategies include selecting access points for each net, sequencing the routing order, and dynamically segregating nets for optimized parallel processing.

3.1.1 Selection of Pin Access Points

Each pin within a net can potentially be connected through multiple access points, presenting a unique challenge in routing optimization. To streamline the net routing process, it's essential to distill these possibilities down to a singular access point for each pin. Our methodology for achieving this involves a calculated approach: first, by determining the centroid of the net's bounding box to establish a focal point, and then, by identifying the nearest access point to this central location. In scenarios where multiple access points are equally proximate to the centroid, we evaluate the accessibility of each, considering the capacity of the grid it occupies. Should these evaluations yield no discernible distinction between access points, a random selection is made from among the candidates. This method ensures a balanced and efficient approach to pin access point selection, significantly simplifying the routing process.

3.1.2 Determining the Order of Nets

The order in which nets are routed critically impacts the efficiency and outcome of the routing process. Through extensive experimentation with various sequencing methods, we have pinpointed an effective heuristic: prioritizing nets based

on their half-perimeter wirelength (HPWL). This strategy favors routing nets with smaller HPWL first, as they typically require fewer resources and are less likely to contribute to congestion. When encounters with ties occur, a random selection is employed to determine the routing sequence. This approach not only simplifies the decision-making process but also significantly enhances the routing algorithm’s ability to manage resources and reduce potential bottlenecks.

3.1.3 Optimized Net Distribution for Parallel Processing

Maximizing the efficiency of parallel processing is paramount in handling complex routing tasks within a reasonable timeframe. To this end, we strategically distribute the nets into eight distinct segments, correlating with the availability of eight CPU threads. This distribution is carefully planned based on the x-coordinate of each net’s bounding box center, ensuring a balanced workload among the segments. Nets that are completely encompassed within the boundaries of a segment are assigned accordingly. Conversely, nets that do not fit neatly into these predefined segments are earmarked for subsequent processing. This methodical distribution facilitates an optimized parallel processing workflow, significantly accelerating the routing process while maintaining high levels of efficiency and workload balance.

By undertaking these comprehensive preprocessing steps, we lay a solid foundation for the routing process, addressing potential challenges upfront and streamlining the path towards achieving optimal routing outcomes.

3.2 Multi-Threaded Pattern Routing

In this phase, we execute the pattern routing process utilizing a parallel approach enabled by multi-threading techniques. For each net, we construct a Steiner tree, leveraging the outcomes to develop the Directed Acyclic Graph (DAG). Within each routing DAG, we identify potential L-shaped or Z-shaped paths connecting every pair of points. Subsequently, we compute the cost associated with each potential path. Utilizing a dynamic programming approach, we assign layers to each vertex. The process culminates in selecting the path that incurs the minimum cost.

3.2.1 Optimization of Wirelength through Steiner Tree Construction

To minimize the wirelength in interconnect routing, the Flute algorithm [5] is employed for the efficient and optimal construction of a Rectilinear Steiner Minimum Tree (RSMT) for configurations involving up to nine pins. This approach is particularly advantageous for reducing the overall wirelength in multi-pin net configurations. An illustrative example is depicted in Figure 3.2, where a Steiner point, labeled 'D', has been strategically added to the net configuration.

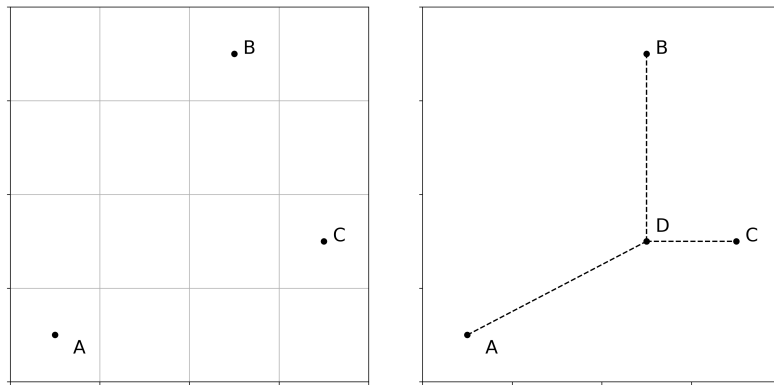


Figure 3.2: (a) A multi-pin net configuration. (b) The obtained optimal RSMT.

3.2.2 Construction of the Routing Directed Acyclic Graph

Upon obtaining the RSMT, we proceed to construct the routing DAG using a depth-first search (DFS) algorithm to traverse the tree's vertices. The traversal sequence is critical, as it defines the directional flow of the edges: for any two vertices connected by an edge, the vertex visited first is designated as the parent, while the subsequent vertex is considered its child. This traversal begins from the vertex with the minimum degree, ensuring an efficient structuring of the routing DAG that reflects the hierarchical relationships within the RSMT. This methodology not only simplifies the representation of the RSMT but also facilitates the optimization of routing paths in subsequent processing stages.

3.2.3 DAG-based Pattern Routing

After constructing a DAG for routing, we establish connections between every pair of points on the same edge through designated geometric patterns. Our methodology introduces two primary shape patterns for this purpose: L and Z. The L shape is predominantly used for the majority of routing scenarios, while the Z shape is reserved for situations that pose routing challenges. To efficiently determine the optimal path, a dynamic programming approach, as detailed in [6], is employed. This involves updating the cost associated with each vertex. Additionally, we apply a strategy to limit the search space by specifically setting the upper and lower layer boundaries, with the goal of identifying the path that minimizes total cost. This composite cost includes three elements: wirelength cost, via cost, and overflow cost.

The overflow cost for a GCell edge, which is dependent on its routing capacity c , routing demand d , and the specific layer l , is calculated using the following

formula:

$$\text{OverflowCost}(c, d, l) = \text{OFWeight}[l] \cdot e^{s(d-c)}, \quad (3.1)$$

where

$$s = \begin{cases} 0.5 & \text{if } c > 0, \\ 1.5 & \text{if } c = 0, \end{cases} \quad (3.2)$$

In this context, s serves as a scaling factor that significantly penalizes the utilization of GCell edges with no capacity by assigning a larger value to s for such cases. The term $\text{OFWeight}[l]$ refers to the overflow weight for GCell edges at the l -th layer, which is defined within the .cap file. This calculation deliberately omits the routing demand overhead that may arise from unstacked vias, under the assumption that the routing demand for GCell edges connected by these vias is already incorporated when managing the connected horizontal and vertical wires.

3.3 Congestion-Aware Pattern Routing

In this stage, our focus shifts towards addressing overflow nets, which significantly impact routing efficiency and overall network performance. To mitigate this, we initially create a congestion map, which is essential for identifying the current routing status. This map is constructed by systematically examining each grid on the graph to pinpoint congested regions accurately. Following the identification process, nets traversing these congested areas are temporarily removed from the routing plan. Subsequently, for the nets affected by this removal, alternative paths or detours are established, specifically for edges that previously passed through congested zones. This approach is inspired by the methodology detailed in [6], which has proven effective in significantly diminishing overflow costs.

3.3.1 Construct Congestion Map

To construct a congestion map, an exhaustive examination of each grid on the graph is performed. During this process, we introduce the concept of overflow thresholds, a pivotal element in congestion analysis. These thresholds are carefully calibrated, allowing us to discern between congested and non-congested areas accurately. By fine-tuning these thresholds, we aim to achieve a balance between reducing the reroute rate and minimizing the impact on the quality of the routing solution. This delicate balance ensures that while congestion is effectively managed, the overall integrity and efficiency of the routing plan are not compromised.

3.3.2 Construct Detours

The detour construction process initiates by assessing the congestion map to pinpoint nets traversing congested areas. For each net, the system seeks out alternate routes, favoring those through less congested zones to ensure efficient rerouting.

The criteria for selecting detours involve a strategic balance between minimizing the detour length and evading new congestion points. This is achieved through a graph-based methodology that dynamically generates detour paths while considering both the present congestion scenario and the spatial limitations of the routing grid. Upon identifying potential detours, we develop a routing DAG and proceed with pattern routing as outlined in Section 3.2, choosing the most cost-effective path for rerouting each net among all alternatives.

Chapter 4

Experimental Results

This chapter presents the performance of our proposed global routing algorithm. We describe the experimental setup, including the computational environment and benchmarks, in Section 4.1. Comparative results, positioning our algorithm against the leading router, GAMER [8], are detailed in Section 4.2, highlighting the efficacy of our approach under identical testing conditions.

4.1 Experimental Setup

We implemented our framework in C++ using the OpenMP library for multi-threading. Experiments were carried out on a Linux system powered by an AMD Ryzen 7 5800X (4.3 GHz) with 32 GB RAM. Contest evaluators validated and scored our routing results.

For benchmarking, we utilized the publicly available ISPD 2024 benchmarks, summarized in Table 4.1.

Design	# std cells	# macros	# nets	# pins	Density (%)	GCell grid
Ariane_sample	122K	133	129K	420K	51	844x1144
MemPool-Tile_sample	129K	20	136K	500K	51	475x644
NVDLA_sample	166K	128	177K	630K	51	1240x1682
BlackParrot_sample	715K	220	770K	2.9M	68	1532x2077
MemPool-Group_sample	3.1M	320	3.3M	10.9M	68	1782x2417
MemPool-Cluster_sample	9.9M	1296	10.6M	40.2M	68	3511x4764
TeraPool-Cluster_sample	49.7M	4192	59.3M	213M	68	7891x10708

Table 4.1: Design statistics of the ISPD 2024 benchmark suite.

4.2 Results and Comparisons

In this section, we evaluate the effectiveness of our routing algorithm in comparison to the established GAMER router. The evaluation is based on a range of benchmarks as detailed in Table 4.2 and Table 4.3.

A comparative analysis of the data indicates that our router consistently performs with lower wire, via, and overflow costs across the majority of the benchmarks. For instance, in the Ariane_sample benchmark, our router achieves a wire cost that is approximately 9% lower than that of GAMER. Similarly, the via cost is lower by 10%, respectively. These improvements in cost are significant, given the complexity of global routing problems.

The trend continues with the BlackParrot_sample and NVDLA_sample, where our router shows a marked reduction in wire costs of about 9% and 8%, respectively. Via costs are also reduced by approximately 6% in both benchmarks. The overflow cost is lower for the BlackParrot_sample but remains comparable for the NVDLA_sample when using our router.

Runtime performance is another critical factor. Our routing algorithm exhibits substantial improvements in runtime across all benchmarks. For instance, our router completes the Ariane_sample benchmark in just 3.48 seconds, which is significantly faster than GAMER’s 57 seconds. This is a remarkable acceleration,

demonstrating not just cost efficiency but also superior computational performance.

However, it is noteworthy that for larger benchmarks, such as the MemPool-Group_sample, while our router maintains lower costs, the runtime is proportionately longer due to the increased problem complexity. For the largest benchmarks, the MemPool-Cluster_sample and TeraPool-Cluster_sample, results from GAMER are not available for comparison.

Overall, our proposed algorithm not only minimizes costs in global routing but also outpaces the benchmark router in terms of runtime, particularly in smaller to medium-sized benchmarks. This performance gain underscores the potential of our algorithm in handling complex global routing challenges efficiently. As future work, we aim to enhance the scalability of our approach to maintain efficiency in larger benchmarks.

Benchmark	Wire Cost	Via Cost	Overflow Cost	Total Cost	Runtime (s)
Ariane_sample	9.30×10^6	3.02×10^6	1.03×10^7	2.26×10^7	3.48
BlackParrot_sample	5.82×10^7	1.95×10^7	3.56×10^7	1.13×10^8	22.69
NVDLA_sample	2.14×10^7	4.48×10^6	2.25×10^7	4.84×10^7	7.66
MemPool-Tile_sample	8.42×10^6	3.39×10^6	3.67×10^6	1.55×10^7	3.71
MemPool-Group_sample	2.64×10^8	7.50×10^7	7.31×10^7	4.12×10^8	137.80
MemPool-Cluster_sample	1.10×10^9	2.64×10^8	3.12×10^8	1.68×10^9	773.95
TeraPool-Cluster_sample	1.23×10^{10}	1.50×10^9	6.69×10^9	2.05×10^{10}	12108.20

Table 4.2: Cost and Running Time of Our Router.

Benchmark	Wire Cost	Via Cost	Overflow Cost	Total Cost	Runtime (s)
Ariane_sample	1.02×10^7	3.35×10^6	1.02×10^7	2.38×10^7	57
BlackParrot_sample	6.40×10^7	2.08×10^7	3.47×10^7	1.19×10^8	1238
NVDLA_sample	2.32×10^7	5.11×10^6	2.22×10^7	5.05×10^7	190
MemPool-Tile_sample	9.22×10^6	3.81×10^6	3.49×10^6	1.65×10^7	43
MemPool-Group_sample	2.91×10^8	8.20×10^7	6.89×10^7	4.42×10^8	4000
MemPool-Cluster_sample	—	—	—	—	—
TeraPool-Cluster_sample	—	—	—	—	—

Table 4.3: Cost and Running Time of GAMER.

Chapter 5

Conclusions and Future Works

In this chapter, we conclude our work in Section 5.1. Next, we provide some directions for future research in Section 5.2.

5.1 Conclusions

Throughout the course of this work, we have not only addressed the global routing problem with an innovative solution, *NTUGR*, but have also gained invaluable insights into the realm of physical design and routing. The development of *NTUGR* stands as a testament to our commitment to minimizing wirelength, via, and overflow costs for large-scale instances, marking a significant milestone in the evolution of global routing technologies.

Our efforts in crafting *NTUGR*, equipped with multi-threading capabilities and compatibility with the simplified formats of the ISPD 2024 Contest [6], have allowed us to efficiently manage scenarios involving more than 200 million pins. By refining the core engine derived from CUGR2 [3] and incorporating advanced patterns while eliminating redundancies, we have achieved unprecedented speed and performance. The novel dynamic net separation framework introduced in this study enhances the router’s efficiency by facilitating effective net distribution and parallel processing.

The experimental outcomes underscore *NTUGR*'s superiority over the current state-of-the-art router, GAMER [8], highlighting significant improvements in terms of wirelength, via count, overflow cost reduction, and computational speed. Beyond these achievements, our journey through the development of *NTUGR* has been richly educational, deepening our understanding of the complexities involved in physical design and routing. This enhanced comprehension has not only contributed to the success of *NTUGR* but has also equipped us with a broader perspective on tackling challenges within the domain.

In conclusion, the development and refinement of *NTUGR* reflect both a substantial leap forward in global routing and a period of profound learning and understanding for our team. As we look to the future, we are eager to apply our expanded knowledge to further advance *NTUGR*, addressing even more complex challenges and continuing to innovate within the field of global routing and physical design.

5.2 Future Works

The exploration of 3D global routing opens several promising directions for future research. We identify two key focus areas:

5.2.1 Pattern Routing

Constructing optimal Rectilinear Steiner Minimum Trees (RSMTs) in 3D space is challenging due to the intricacies of layer assignment, congestion management, net segmentation strategies, and the optimization of net routing sequences. Our preliminary dynamic programming solution, inspired by previous work [3], requires further development. Notably, addressing congestion issues extends beyond utilizing flute [1] for RSMT estimation. A prospective enhancement involves employing reinforcement learning to dynamically predict RSMTs, factoring in congestion levels to improve routing decisions and efficiency.

5.2.2 Maze Routing

Maze routing, while accurate, is notably omitted from our current routing algorithm due to its high computational cost. We plan to address this by considering a hybrid strategy that reduces the search space, as suggested in [12], and by exploring the potential of GPU-accelerated maze routing [8]. Implementing an efficient data structure for grid management will be crucial for this advancement, allowing for parallelization and maintaining the quality of the routing solution. Optimization of the search space will also be critical to ensure that the accelerated maze routing is both fast and accurate.

Bibliography

- [1] C. Chu and Y.-C. Wong, “FLUTE: Fast Lookup Table Based Rectilinear Steiner Minimal Tree Algorithm for VLSI Design,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 1, pp. 70–83, 2007.
- [2] G. C. J. Liu and E. F. Y. Young, “REST: Constructing Rectilinear Steiner Minimum Tree via Reinforcement Learning,” in *Proceedings of ACM/IEEE Design Automation Conference*, San Francisco, CA, USA, December 2021.
- [3] L. Jinwei and E. F. Young, “EDGE: Efficient DAG-based Global Routing Engine,” in *Proceedings of ACM/IEEE Design Automation Conference*, San Francisco, CA, USA, July 2023.
- [4] E. F. Y. Jinwei Liu, “CUGR: Detailed-Routability-Driven 3D Global Routing with Probabilistic Resource Model,” in *Proceedings of ACM/IEEE Design Automation Conference*, San Francisco, CA, USA, July 2020.
- [5] R. Kastner, E. Bozorgzadeh, and M. Sarrafzadeh, “Pattern routing: Use and theory for increasing predictability and avoiding coupling,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 21, no. 7, pp. 777–790, 2002.
- [6] R. Liang, A. Agnesina, W.-H. Liu, and H. Ren, “GPU/ML-Enhanced Large

- Scale Global Routing Contest,” in *Proceedings of ACM International Symposium on Physical Design*, Taipei, Taiwan, March 2024.
- [7] S. Lin and M. D. F. Wong, “Superfast Full-Scale GPU-Accelerated Global Routing,” in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, San Diego, CA, USA, October 2022.
- [8] S. Lin, J. Liu, E. F. Young, and M. D. Wong, “GAMER: GPU-Accelerated Maze Routing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 2, pp. 583–593, 2023.
- [9] W.-H. Liu, W.-C. Kao, Y.-L. Li, and K.-Y. Chao, “NCTU-GR 2.0: Multi-threaded collision-aware global routing with bounded-length maze routing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 32, no. 5, pp. 709–722, 2008.
- [10] —, “Multi-threaded collision-aware global routing with bounded-length maze routing,” in *Proceedings of ACM/IEEE Design Automation Conference*, Anaheim, CA, USA, June 2010.
- [11] M. Pan and C. Chu, “Fastroute: A step to integrate global routing into placement,” in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, San Jose, CA, USA, June 2006.
- [12] —, “FastRoute 2.0: A High-quality and Efficient Global Router,” in *Proceedings of IEEE/ACM International Conference on Computer-Aided Design*, Yokohama, Japan, January 2007.
- [13] Z. Zhou, Y. Pan, G. G. Lemieux, and A. Ivanov, “MEDUSA: A Multi-resolution Machine Learning Congestion Estimation Method for 2D and 3D Global Rout-

ing,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 28, no. 5, pp. 1–25, 2023.